Università degli Studi di Napoli Federico II

Scuola Politecnica e delle Scienze di Base

Dipartimento di Ingegneria Elettrica e Tecnologie dell'Informazione

Corso di Laurea Magistrale in Ingegneria Informatica

MA CH STAI DICENN
PARLA CHIARO: A Dialect-Aware Misunderstanding Alert System for AI Pipelines on Italian Regional Speech

a.a. 2025-26

Studenti:

D'Avino Matteo M63001724

Fienga Luigi M63001733

Roscigno Andrea M63001778

Indice

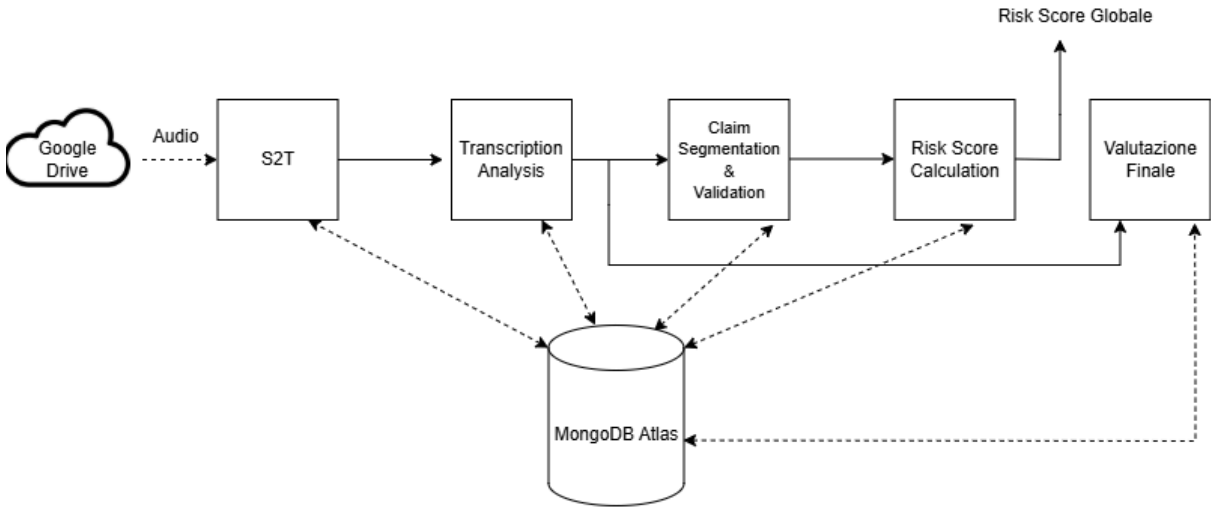
Introduzione al problema..... 3
1. Pipeline proposta..... 4
2. MongoDB Atlas..... 5
3. S2T (Speech to text)..... 13
4. Transcription Analysis..... 17
5. Claim Segmentation..... 20
6. Risk Score Calculation..... 22
7. Metriche e risultati..... 26
8. Validazione Whisper RAG e Risk Score..... 29
8.1 Validazione Whisper RAG..... 29
8.2 Validazione modulo risk score..... 30
9. Dashboard..... 32
10. Trascrizione in tempo reale..... 34
11. Migliorie per il futuro..... 35

Introduzione al problema

L'Italia presenta un panorama linguistico unico, caratterizzato da decine di varietà regionali e dialetti parlati quotidianamente da milioni di persone. Questa ricchezza culturale rappresenta tuttavia una notevole sfida tecnologica, poiché gli attuali sistemi di Riconoscimento Vocale Automatico (ASR) sono addestrati prevalentemente sulle varianti linguistiche standard. Quando un utente interagisce con un assistente vocale utilizzando espressioni dialettali o alternando dialetto e italiano (*code-switching*), l'ASR introduce inevitabilmente errori, sostituzioni fonetiche e omissioni nella trascrizione. Questa problematica viene amplificata negli scenari in cui le trascrizioni distorte vengano passate silenziosamente agli LLM, che le interpreta con ingiustificata sicurezza e produce risposte potenzialmente fuorvianti senza avvertire minimamente l'utente del malinteso in corso.
Per provare a risolvere questo problema, il seguente progetto si è occupato di sviluppare ***Dialect-Aware Warning System*** **(DAWS),** un framework adattabile a qualunque dialetto del nostro paese per il corretto riconoscimento di parole mal scritte all'interno di trascrizioni dei sistemi SAR per audio dove si parla in dialetto o in dialetto/italiano. L'output finale del sistema sarà un valore di risk score globale su tutta la trascrizione, Il lavoro si è concentrato sul dialetto napoletano, ma niente vieta con le opportune modifiche per il dataset di partenza nel utilizzarlo anche per gli altri dialetti della nostra penisola.
Le tecnologie utilizzate all'interno del progetto sono le seguenti:

- Google Drive, usato come locazione di memoria del sample di audio fornito per valutare la pipe
- Whisper, serve per trascrivere i vocali ottenendo frasi semanticamente e sintatticamente valide.
- PhoneticXeus, permette di ottenere una trascrizione del vocale con il linguaggio fonetico universale (IPA)
- Mistral-large-3-675B-instruct-2512, è il modello usato per l'analisi delle trascrizioni e la classificazione dei tipi di claim. Il seguente modello è stato anche validato per la comprensione del napoletano.
- spaCy è una libreria di python per il NLP, servito per il modulo di Claim Segmentation,
- dbmdz/bert-base-italian-cased è un modello bert-like utilizzato per la valutazione del senso semantico delle frasi tradotte in italiano, mediante la metrica del PPPL.
- MongoDB Atlas, serve per memorizzare tutti i risultati partendo dagli step intermedi del sistema fino al risultato finale.

1. Pipeline proposta



Il sistema proposto è composto da quattro moduli principali, ognuno addetto ad un ruolo specifico per la corretta implementazione della pipe.

Google Drive è stato utilizzato per la memorizzazione degli audio da analizzare**.**

Il modulo **S2T** (Speech to Text) contiene al suo interno un modulo PhoneticXeus e un modulo Whisper con tecnica RAG, quest'ultimo verrà approfondito nel capitolo successivo, ma l'idea è quella di voler fornire a Whisper un ***"initial prompt"***, come contesto iniziale, per agevolare la trascrizione del medesimo garantendo anche di trascrivere in maniera parzialmente corretta le parole in napoletano. Il modulo quindi riceve il file audio da trascrivere e produce in uscita: una trascrizione IPA, la trascrizione sporca di Whisper usata per la ricerca nel RAG, una trascrizione mediante il sistema RAG, e i livelli di confidenza della trascrizione di Whisper RAG per ogni token, che poi sono stati calcolati a livello di parole.

Il **Transcription Analysis** riceve in ingresso i risultati di S2T, mediante l'uso di mistral-large-3, restituisce la traduzione in italiano della frase trascritta sulla base dell'output di S2T e le parole risolte/problematiche, di questa frase tradotta verrà valutato anche il PPPL grazie a bert-base-italian-cased.

Claim segmentation & Validation si occupa invece di dividere il trascritto normalizzato in italiano in tanti claim quanti se ne individuano, dove per claim si intende la più piccola unità di testo che ha senso compiuto. In seguito alla loro determinazione, i claim vengono poi validati medianti un LLM che fornisce informazioni di contesto come il tipo di claim scelto tra un set di possibilità (domanda generale, richiesta farmaco…) e le entità intorno alle quali gira l'intero claim (nome di un farmaco, un sintomo…). Una volta fatto ciò mediante sempre allo stesso LLM viene chiesto di mappare il trascritto napoletano alla traduzione italiana, infine l'ultima cosa è il calcolo del PPPL per claim.

Risk Score Calculation è il modulo che si occupa del calcolo del risk score globale, il quale si determina mediante un insieme di regole, riconoscendo la prima che soddisfa un criterio di

specificità, queste regole sono basate sulla distribuzione dei tipi di rischi e sui tipi di claim sul periodo. Il risk score per claim è invece valutato in base anche ad una combinazione lineare tra risk score dato dalla trascrizione e quello dato dalla traduzione; il fattore alpha che lega i due termini è definito in base al tipo di claim. Il risk score di trascrizione è ancora una combinazione lineare tra una matrice di pesi (definita per ogni categoria) e metriche caratteristiche di quel claim come la confidenza minima tra le parole del claim, la confidenza media ed altre. Il risk score per la traduzione dipende solo da PPPL normalizzato tra 0 e 1.

Valutazione Finale Questo modulo ha l'obiettivo di analizzare l'efficacia delle traduzioni generate in *Transcription Analysis* e di confermare la validità dell'integrazione di *Whisper RAG*. Le valutazioni prestazionali si avvalgono di metriche standard quali cosine-similarity, BLEU, ROUGE e WER. Per quanto riguarda invece la validazione del calcolo del Risk Score, questa viene effettuata su un campione ridotto di messaggi vocali analizzati manualmente, ai quali viene attribuito un punteggio di riferimento in una scala da 0 a 5.

MongoDB Atlas è l'infrastruttura che funge da deposito centralizzato sia per i metadati relativi ai file audio e ai partecipanti coinvolti, sia per i dati intermedi generati dai vari stadi della pipeline. L'obiettivo finale è il consolidamento di tali informazioni in un documento sintetico all'interno della collezione Sessions, che rappresenta l'esito conclusivo dell'intero processo di elaborazione.

2. MongoDB Atlas

L'adozione di MongoDB Atlas è risultata la scelta più coerente con la natura degli artefatti generati dal sistema, costituiti prevalentemente da dizionari che si integrano nativamente con il modello documentale di MongoDB. Un vantaggio determinante è rappresentato dalla flessibilità strutturale dei documenti, che consente una gestione dinamica e adattabile delle collezioni senza i vincoli tipici delle architetture relazionali; tale caratteristica è fondamentale in un progetto dove la soluzione si evolve progressivamente attraverso numerose modifiche in corso d'opera. Inoltre, in un'ottica di scalabilità su grandi volumi di dati, le strutture relazionali avrebbero introdotto eccessive latenze nel recupero delle informazioni, specialmente a causa della complessità delle operazioni di join tra tabelle.

Un ulteriore pilastro metodologico nell'impiego di MongoDB Atlas risiede nella possibilità di unificare, all'interno di un'unica infrastruttura, sia il database documentale che quello vettoriale, semplificando sensibilmente la gestione degli embedding necessari per l'architettura RAG.

Altre soluzioni come Kafka o Spark non sarebbero state utili in quanto non si è lavorato su streaming in tempo reale, oltretutto non si presenta quello che è uno schema event-driven. Kafka e Spark sono strumenti per pipeline di streaming ad alto throughput su volumi di dati continui e distribuiti. Il caso d'uso qui è radicalmente diverso: ogni sessione è un evento singolo, pesante in termini di elaborazione GPU (trascrizione Whisper, analisi LLM, ensemble) ma raro in termini di frequenza. Non c'è uno stream continuo di eventi da processare in parallelo su cluster distribuiti. Usare Kafka come bus di messaggi per orchestrare gli stadi della pipeline sarebbe un'architettura sovradimensionata: introdurrebbe complessità operativa (broker, consumer groups, partition management) per un problema che si risolve con una sequenza sincrona di chiamate a funzione dentro un notebook Kaggle. Analogamente, Spark ha senso quando i dati non entrano in memoria su una singola macchina.

Neo4j non è appropriato in quanto non ci sono dati che creano strutture sociali o relazionali, non ci sono grafi di relazioni da attraversare, nessuna query del tipo "trova tutte le sessioni il cui partecipante conosce qualcuno che ha avuto lo stesso pattern di claim". Neo4j porterebbe overhead di modellazione (definire nodi, archi, proprietà) senza nessun vantaggio interrogativo rispetto a un $lookup MongoDB.

Anche Redis non è appropriato usarlo perché non c'è nessuna necessità di integrare meccanismi di caching. Esso è un key-value store in memoria, progettato per accesso ultra-rapido a dati semplici. Non ha capacità di query strutturate, aggregation pipeline, proiezioni o join. Sarebbe adeguato come layer di cache sopra MongoDB (per evitare query ripetute), ma non come database primario: non permette di interrogare "tutte le sessioni completate con risk level red raggruppate per dialetto" senza caricare tutto in memoria e filtrare nel codice applicativo.

Di seguito si riportano i diagrammi delle collezioni presenti su MongoDB Atlas

Sessions

_id: *String (partecipand_id + file_name)*

partecipandId: *String (Ref: Partecipants)*

promptId: *UUID (Ref: Recordings)*

promptCategory: *String (Ref: Recordings)*

promptText: *String (Ref: Recordings)*

startedAt: *Unix timestamp in ms*

status: *String (completed/failed/processing)*

completedAt: *Unix timestamp in ms*

overall_risk: *Object*
↳ **overall_risk_score**: *Double (Ref: Risk_Logs)*
overall_risk_level: *Double (Ref: Risk_Logs)*

error_log: *String (optional)*

Recordings

_id: *String (partecipand_id + file_name)*

partecipandId: *String*

promptId: *UUID*

promptText: *String*

promptCategory: *String*

recordingIndex: *Integer*

filename: *String*

size: *Integer*

mimetype: *String*

uploadedAt: *String*

drive_file_id: *String*

drive_folder_id: *String*

Translation_Metrics

_id: *String (partecipand_id + file_name)*

started_at: *Unix timestamp in ms*

saved_at: *Unix timestamp in ms*

cosine_similarity: *Double*

bleu: *Double*

rouge1: *Double*

rouge2: *Double*

rougeL: *Double*

wer: *Double*

prompt_text: *String*

translated_text: *String*

embedding_model: *String*

Dialect_KnowledgeBase

_id: *Long*

napoletano: *String*

italiano: *String*

campo: *String*

parole_chiave: *Array*
↳ **i-esimo:** *String*

embedding: *Array (Dim. 384)*
↳ **i-esimo:** *Double*

Partecipants

_id: *String*

gender: *String*

ageRange: *String*

dialect: *String*

dialectOther: *String*

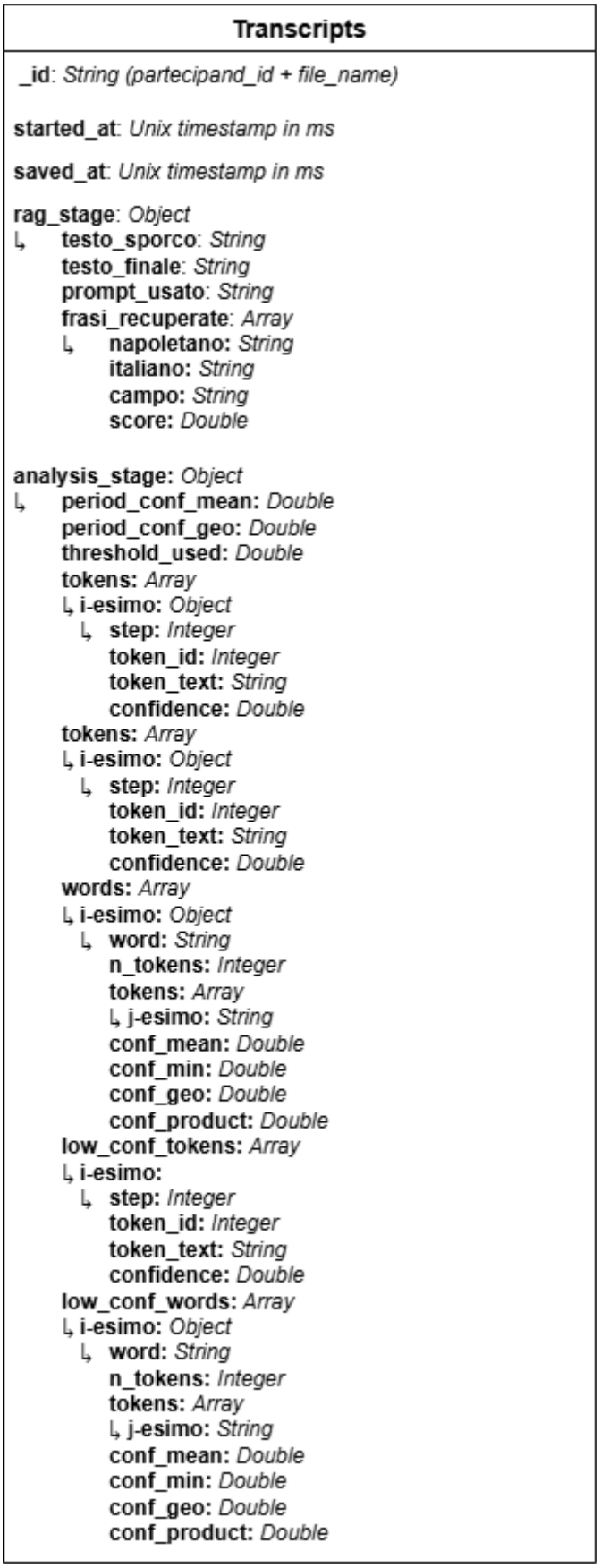
education: *String*

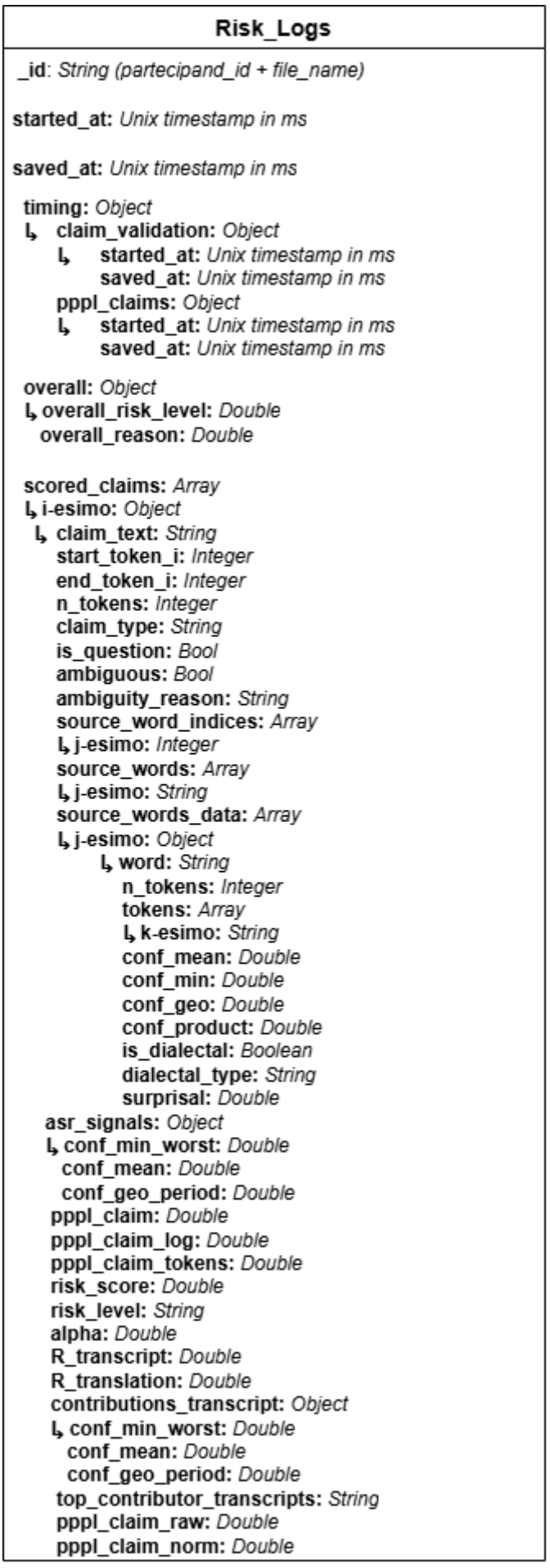
livingContext: *String*

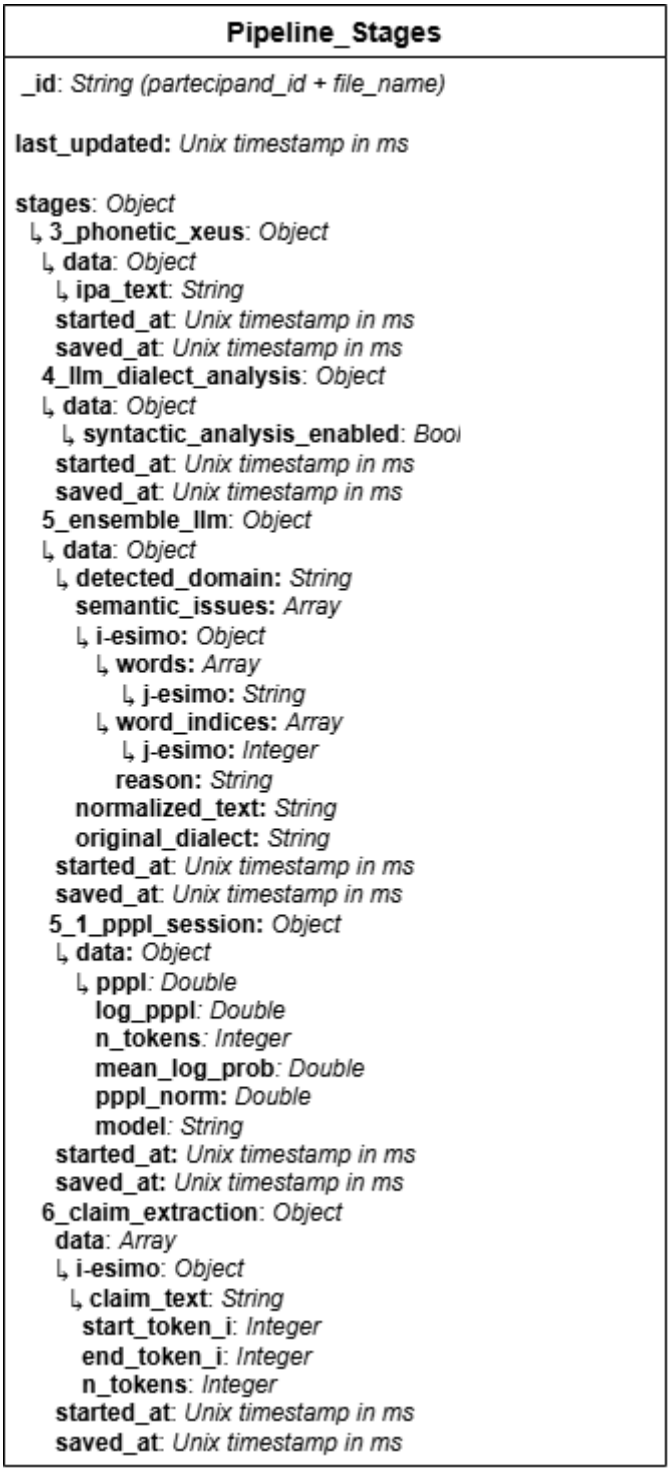
dialectFrequency: *String*

createdAt: *String*

partecipandId: *String*


Transcripts
_id: String (partecipand_id + file_name)
started_at: Unix timestamp in ms
saved_at: Unix timestamp in ms
rag_stage: Object
testo_sporco: String
testo_finale: String
prompt_usato: String
frasi_recuperate: Array
napoletano: String
italiano: String
campo: String
score: Double
analysis_stage: Object
period_conf_mean: Double
period_conf_geo: Double
threshold_used: Double
tokens: Array
i-esimo: Object
step: Integer
token_id: Integer
token_text: String
confidence: Double
tokens: Array
i-esimo: Object
step: Integer
token_id: Integer
token_text: String
confidence: Double
words: Array
i-esimo: Object
word: String
n_tokens: Integer
tokens: Array
j-esimo: String
conf_mean: Double
conf_min: Double
conf_geo: Double
conf_product: Double
low_conf_tokens: Array
i-esimo:
step: Integer
token_id: Integer
token_text: String
confidence: Double
low_conf_words: Array
i-esimo: Object
word: String
n_tokens: Integer
tokens: Array
j-esimo: String
conf_mean: Double
conf_min: Double
conf_geo: Double
conf_product: Double


Risk_Logs
_id: String (partecipand_id + file_name)
started_at: Unix timestamp in ms
saved_at: Unix timestamp in ms
timing: Object
claim_validation: Object
started_at: Unix timestamp in ms
saved_at: Unix timestamp in ms
pppl_claims: Object
started_at: Unix timestamp in ms
saved_at: Unix timestamp in ms
overall: Object
overall_risk_level: Double
overall_reason: Double
scored_claims: Array
i-esimo: Object
claim_text: String
start_token_i: Integer
end_token_i: Integer
n_tokens: Integer
claim_type: String
is_question: Bool
ambiguous: Bool
ambiguity_reason: String
source_word_indices: Array
j-esimo: Integer
source_words: Array
j-esimo: String
source_words_data: Array
j-esimo: Object
word: String
n_tokens: Integer
tokens: Array
k-esimo: String
conf_mean: Double
conf_min: Double
conf_geo: Double
conf_product: Double
is_dialectal: Boolean
dialectal_type: String
surprisal: Double
asr_signals: Object
conf_min_worst: Double
conf_mean: Double
conf_geo_period: Double
pppl_claim: Double
pppl_claim_log: Double
pppl_claim_tokens: Double
risk_score: Double
risk_level: String
alpha: Double
R_transcript: Double
R_translation: Double
contributions_transcript: Object
conf_min_worst: Double
conf_mean: Double
conf_geo_period: Double
top_contributor_transcripts: String
pppl_claim_raw: Double
pppl_claim_norm: Double



Si specifica come tutti i moduli internamente alla pipe memorizzino correttamente il timestamp di inizio e fine processo.

Sessions è la collezione che tiene traccia delle sessione dell'audio da processare; L'id univoco è assegnato come la combinazione dell'id univoco del partecipante e del nome del file della registrazione, così come l'id di **Transcripts, Pipeline_Stages, Recordings e Translation_Metrics.** Lo status può assumere valore *completed* se correttamente processato il file, *failed* se non si è completato e ha generato errori o *processing* nel caso in

cui l’audio sia in elaborazione; alcuni campi qui presenti sono copiati da Recordings o dai risultati overall del nodo del calcolo di Risk.

Transcripts, contiene tutte le informazioni in merito alle trascrizioni effettuate da Whisper, in più ci sono anche i livelli di confidenza a livello di token e quelli calcolati a livello di parole.

Risk_Logs memorizza tutte le informazioni in merito alla divisione in claim del periodo tradotto in italiano, la mappatura dei claim tra l’italiano e il trascritto semi napoletano, ed infine tutti i risultati in merito al calcolo dei risk score sia a livello di claim che a livello globale.

Pipeline_Stages, contiene tutte le informazioni prodotte nei restanti stadi della Pipe al di fuori di quelle già presentate; ovviamente i timestamp salvati qui sono anche sugli step intermedi.

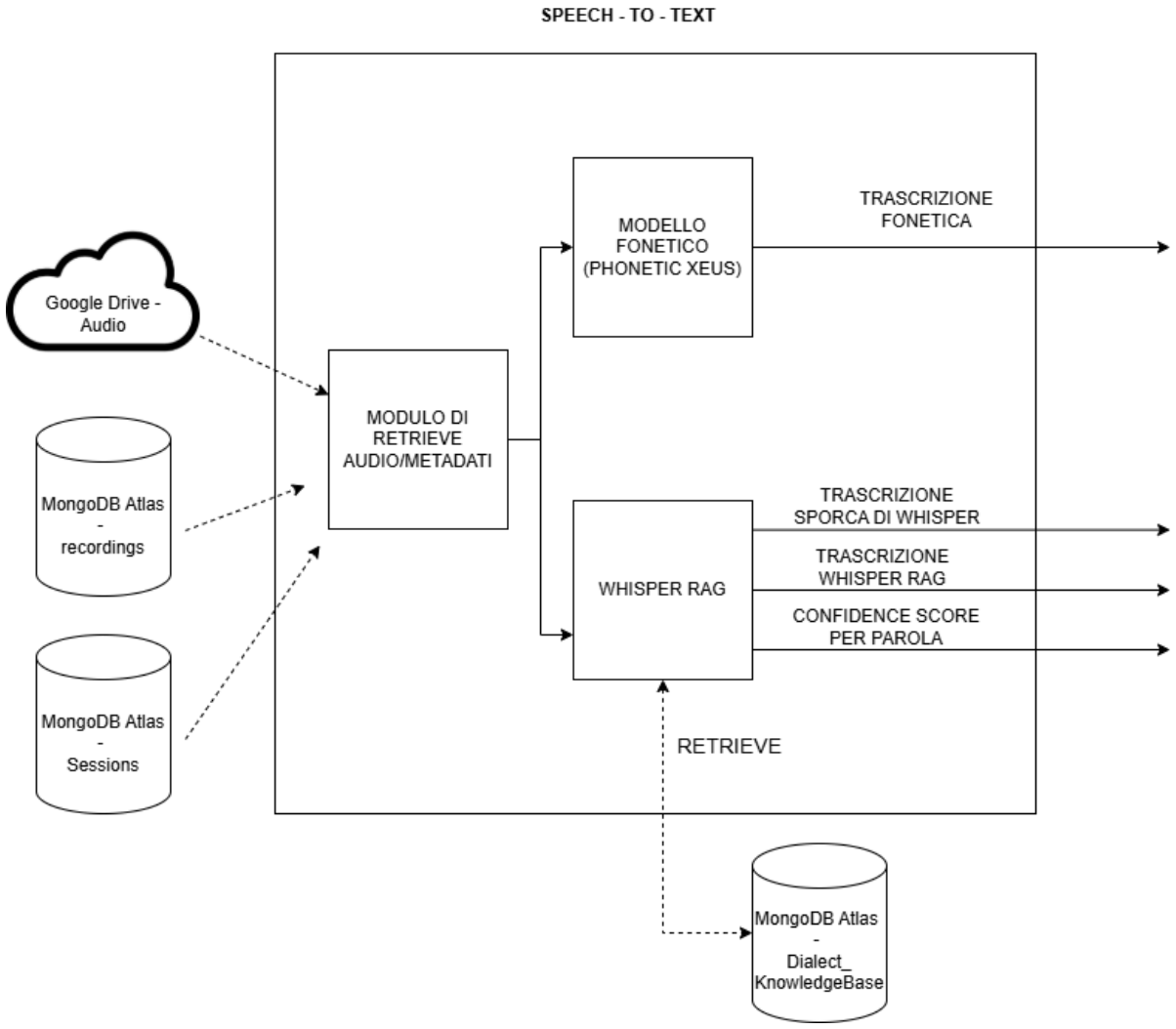
Recordings contiene la struttura dei metadati forniti per ogni registrazione.

Partecipants contiene i metadati dei partecipanti dal campione fornito.

Translation_Metrics, memorizza i risultati di metriche di interesse per valutare quanto è performante la pipe a livello dei moduli S2T e Transcription Analysis, le metriche scelte sono: cosine_similarity, BLUE, ROUGE-1, ROUGE-2 ROUGE-L e WER.

La collezione **Dialect_KnowledgeBase** contiene il dizionario delle frasi napoletane per il sistema RAG, come verrà detto successivamente gli embedding sono calcolati sulle frasi tradotte in italiano. Inoltre qui è presente un solo documento raccogliendo particelle e verbi comuni napoletani per il fuzzy matching.

3. S2T (Speech to text)



Entrando nel dettaglio del modulo S2T, la prima cosa che viene effettuata è il retrive dei vocali che si vogliono processare sulla base dei metadati presenti nella collezione ***recordings***, partendo da questi ultimi si vanno a prelevare solo gli audio da google drive che non sono stati ancora processati o che sono già presenti nel DB ma hanno status failed o processing; in quest'ultimo caso si cancellano tutti i documenti tra le varie collezione che hanno lo stesso *_id*, per eliminare eventuali refusi.
Una volta prelevati gli audio, un per volta vengono passati in ingresso a PhoneticXeus il quale ci restituisce una trascrizione nel linguaggio IPA come ad esempio:

Frase Originale: *Il mio cuore inizia improvvisamente a battere molto velocemente da tre anni. Ho fatto un'ecografia al cuore e un controllo con gli elettrodi e sono sempre risultati normali. Ultimamente sento dolore al petto e un formicolio nel braccio destro. Anche la tiroide è normale. Non so cosa fare.*

PhoneticXeus:
okoramikomint͡ʃaabaterealintrasatmultovelot͡ʃementeatreanad͡ʒifatenekongrafiookoraenunkon

trolkueletrodiɛnonsemprsulrizultatonormalemɔsentenonuloronpieteenunformikoliobrat͡ʃodestr opuratirojdeenormalelunsert͡ʃolead͡ʒafa

Il meccanismo di *“Whisper RAG”* rappresenta una soluzione progettuale di rilievo, concepita per superare i limiti intrinseci del comportamento standard di Whisper. In assenza di configurazioni specifiche, il modello tende a trascrivere gli audio applicando forzatamente le regole sintattiche e semantiche della lingua rilevata. Nel contesto dei dialetti o del linguaggio misto, questa caratteristica causa una sistematica “italianizzazione” dei termini: Whisper può generare parole inesistenti nel dizionario o, in casi ancora più critici, alterare completamente il senso originale trasformando espressioni napoletane in frasi italiane sintatticamente corrette ma semanticamente distanti dalla realtà. Tale distorsione comprometterebbe l'intera affidabilità della pipeline, rendendo necessari gli interventi strutturali descritti in seguito.

Il processo si basa sul campo *prompt_ids* di Whisper, che permette di dare un contesto alla trascrizione da effettuare. Tale campo viene riempito con al più 4 frasi che sono verosimilmente simili all’audio da trascrivere e una serie di parole napoletane concatenate che sono simili a quelle presenti in una trascrizione sporca. Per realizzare questo processo è stato necessario realizzare un dizionario di frasi napoletane, il quale ha oggetti con la seguente struttura:

```
  {
    "napoletano": "Dotto', tengo nu forte dulore 'e cape e nun me sento bbuono.",
    "italiano": "Dottore, ho un forte mal di testa e non mi sento bene.",
    "campo": "salute",
    "parole_chiave": ["dulore", "cape", "tengo", "bbuono"]
  }
```

Per le parole extra è stato costruito un dizionario di particelle e di verbi:

```
 "verbi_comuni": [
    "teng",
    "stong",
    "sent",
    "sento",
    "vac",
    "fann",
    "bruc",
…
```

"particelle": [

"'nu poc",

"tutt",

"doie",

"chin",

"nguoll",

…

Il prompt ha un numero massimo di token che possono essere dati in ingresso pari a 224 tokens, nel caso in cui si superasse quel limite viene troncato il prompt.

Per la corretta costruzione del prompt il processo di RAG avviene come segue:

- Avviene una prima trascrizione *sporca* mediante whisper-large-v3 dove avremo una trascrizione italianizzata e parzialmente sbagliata della nostra frase
- Su questa frase viene calcolato un vettore di embedding tramite paraphrase-multilingual-MiniLM-L12-v2
- A partire da questo embedding viene ricercato nello spazio vettoriale della base di conoscenza costruita su MongoDB Atlas mediante ANN (approximate nearest neighbor), vengono restituiti 4 vettori che somigliano di più alla trascrizione sporca.
- Il recupero dei termini napoletani avviene attraverso una metodologia di ricerca fuzzy supportata dalla libreria rapidfuzzy. Prima di attivare gli algoritmi della libreria, il sistema verifica l'esistenza di una corrispondenza esatta tra le parole della trascrizione e i lemmi presenti nel dizionario napoletano (solo verbi_comuni e particelle); solo in assenza di un match diretto si procede con rapidfuzzy. Lo strumento analizza la somiglianza tra le stringhe carattere per carattere, restituendo un punteggio espresso in centesimi. Qualora tale valore sia superiore alla soglia critica di 72, il termine napoletano identificato come più affine viene validato e accettato dal sistema.
- Infine il prompt finale viene costruito mediante la concatenazione delle 4 frasi e le parole napoletane recuperate con la logica fuzzy.

Ovviamente c'è da specificare che il retrieval sulla base di conoscenza degli embedding in Atlas è composta dalle frasi italiane, da cui poi ci verrà restituito tutto il documento memorizzato da cui si prelevano le frasi napoletane. La trascrizione finale è realizzata con *whisper-large-v3* mediante l'uso del prompt costruito.

Frase Originale: *Il mio cuore inizia improvvisamente a battere molto velocemente da tre anni. Ho fatto un'ecografia al cuore e un controllo con gli elettrodi e sono sempre risultati normali. Ultimamente sento dolore al petto e un formicolio nel braccio destro. Anche la tiroide è normale. Non so cosa fare.*

Prompt: *Me manca 'o sciato e tengo nu dulore vicino 'o core. Dottore m sent i recchie tutt applat Dotto' m vulit scriver doie 'nals, non mi sento tanto buono Dottò a nu poc a sta vij*

m'aggir a cap 'nu poc tutt doie chin nguoll spiss spess appicciat applat scassat ultimament 'o 'a 'e nu na ll' si' mo' ca pe' 'ncopp 'mmiezo teng stong sent sento vac fann bruc pigliat acal m'avot ammsuratam vulit scriver fa mal fann mal m fa mal m sent m bruc tengo tene tenimmo pozzo po' potimmo stongo sta stammo aggio ha avimmo vaco va iammo saccio sa sapimmo voglio vo' vulimmo dico dice dicimmo faccio fa facimmo fann pur. dulore fatto cosa cuore piede faccio sento

Testo Sporco: *Il mio cuore comincia a battere all'intrasato molto velocemente, a tre anni. Ho fatto un'icografia del cuore e un controllo con elettrodi e non sempre sul risultato normale. Ora sento un olore, un piede e un formicolio sul braccio destro. Pure la tiroide è normale, non so cosa faccio.*

Testo Finale: *O core mi comincia a battere all'intrasat molto velocemente, a tre anni. Aggi fatte necografia o core e nu control cu elettrodi e non sempre sul risultato normale. Mo' sento nu ulore in piet e nu formicolio u braccia destra. Pura tiroide e normale, nun saccio che aggia fa.*

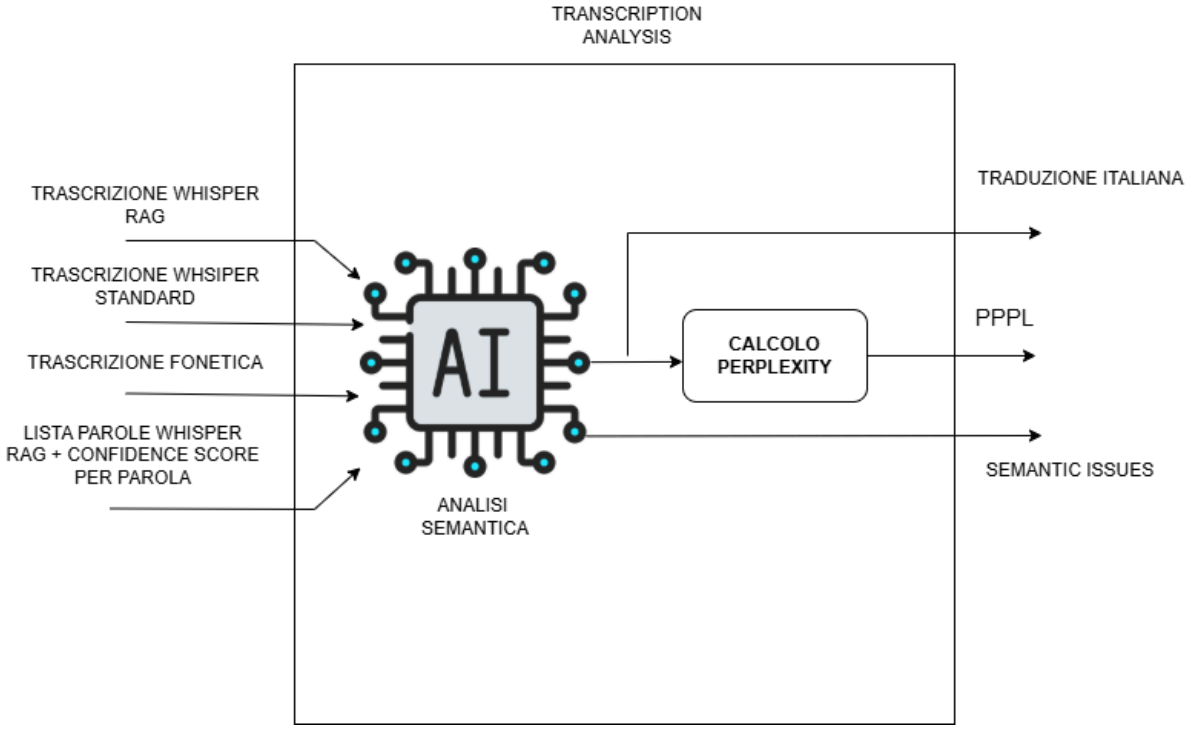
Una volta effettuata la trascrizione vengono calcolate le confidence per token. Il modello fornisce per ogni token dei logits per tutti quelli possibili del suo vocabolario, dopo di che si fa una normalizzazione passandole a delle log-probabilità dove 0 rappresenta che whisper è molto sicuro della parola trascritta, valori negativi rappresentano che si sente meno sicuro, per essere più leggibili si passano ad un esponenziale per portale tra 0 e 1.

Da questi valori poi vengono calcolate le confidence, calcolando valori su singola parola come:

- numero di token che compongono la parola
- il valore medio di confidenza tra token
- la confidenza minima tra i token di quella parola
- la media geometrica tra le confidenze dei token
- i prodotti delle confidenze tra token

L’output finale di questo modulo sarà quindi la trascrizione mediante tecnica RAG, la trascrizione sporca, le confidenze delle parole sul primo testo e la trascrizione IPA.

4. Transcription Analysis



L'obiettivo del modulo di analisi delle trascrizioni è effettuare, mediante l'uso di un LLM, un'analisi semantica dell'audio trascritto, usando tutte le sorgenti di dati disponibili, ovvero trascrizione Whisper standard, trascrizione Whisper RAG, confidence score parole Whisper RAG e trascrizione di PhoneticXeus, per generare una traduzione semanticamente coerente in italiano ed una lista di parole/espressioni ritenute problematiche.

Per la scelta di quale LLM utilizzare a tale scopo si è pensato di effettuare un test di validazione su un piccolo sottoinsieme di 20 frasi in napoletano estratte casualmente dal sistema RAG, che coprono in maniera uniforme i vari topic della collezione memorizzata. L'obiettivo della validazione è constatare la comprensione del dialetto napoletano da parte di tre modelli differenti: **Mistral-large-3-675B-instruct-2512**, **GPT-oss-120b** e **Gemma-3n-e4b-it**.
Per valutare quindi i suddetti modelli si è chiesto loro di rispondere, in italiano, a diversi messaggi scritti in dialetto napoletano, specificando anche quale dialetto fosse stato utilizzato. In base alle risposte fornite, si è assegnato loro uno score da 0 a 5 come punteggio di coerenza. In sintesi questo test vuole valutare se un modello ha la conoscenza della lingua napoletana, che è uno step in più rispetto ad un semplice task di traduzione.
Effettuando il test si è notato che:

- mistral-large-3-675b-instruct-2512 ha avuto uno score medio di 3.85, riuscendo a mantenere coerenza nelle diverse risposte;
- gpt-oss-120b ha avuto uno score medio pari a 3.05, con una tendenza a troncare le risposte fornite in uscita;
- gemma-3n-e4b-it ha avuto uno score medio pari a 2.7, con una qualità generale delle risposte più bassa rispetto agli altri due.

Da tale confronto è emerso, quindi, che il modello ottimale è mistral-large, che difatti è stato il LLM utilizzato all'interno della pipe, anche grazie ad una finestra di contesto maggiore rispetto agli altri, necessaria in quanto la quantità di token di input e output è considerevole nelle diverse chiamate.

Implementazione analisi

Tutta l'analisi viene effettuata dal modello all'interno di una singola chiamata, in cui il compito ad esso richiesto viene suddiviso in tre fasi sequenziali:

- identificazione del dominio di riferimento (es. medico o quotidiano);
- validazione semantica, effettuata incrociando i dati di trascrizione con evidenze fonetiche (IPA);
- la traduzione in un italiano standard e fluido senza alterare il significato originale.

In particolare, la trascrizione ottenuta tramite Whisper-RAG costituisce la base principale dell'analisi, in quanto beneficia delle informazioni recuperate dal sistema di Retrieval-Augmented Generation e risulta generalmente più accurata nella ricostruzione del testo. La trascrizione originaria prodotta da Whisper viene invece utilizzata come riferimento secondario per evidenziare eventuali differenze introdotte dal processo di retrieval e per verificare se alcune parole siano state modificate in modo significativo tra le due versioni. A queste si aggiunge la trascrizione fonetica in formato IPA generata da PhoneticXeus, che permette al LLM di valutare la plausibilità fonetica di termini particolarmente incerti o di difficile interpretazione. Il system prompt istruisce esplicitamente il modello a non considerare nessuna di queste fonti come assolutamente corretta, ma a integrarle criticamente insieme alle confidence score e al contesto semantico globale della frase. Ogni sorgente disponibile ha un punto debole specifico. Whisper-RAG, arricchito con il vocabolario napoletano/campano, tende a essere contaminato dal prior lessicale: può copiare parole dialettali suggerite dal RAG anche quando l'audio non le supporta acusticamente. Whisper Standard, senza priors, fallisce spesso l'italianizzazione di fonemi dialettali producendo output semanticamente insensati. PhoneticXeus fornisce una trascrizione IPA language-independent, quindi immune ai bias lessicali, ma non è allineata posizionalmente e può fondere parole adiacenti in sequenze continue. Nessuna delle tre è sufficiente da sola: solo la triangolazione tra tutte e tre permette di distinguere un errore da una forma dialettale legittima. Il prompt è diviso in due parti: un system_prompt fisso e un user_prompt dinamico che inietta le quattro sorgenti concrete per ogni sessione.

In particolare, l'architettura del system prompt sfrutta diverse fasi per avvicinare quanto più possibile il LLM ad uno strumento di analisi quasi deterministico, riducendo drasticamente lo spazio latente delle allucinazioni. Nello specifico, il **Role Prompting** non si limita ad assegnare una "persona", ma condiziona la rete semantica del LLM a dare priorità a euristiche incrociate di linguistica dialettale e rigore clinico; questa postura viene potenziata dalla **Task Decomposition**, che costringe il modello a frammentare il ragionamento (sfruttando una forma implicita di *Chain of Thought*). Sul fronte della sicurezza dell'output, il **Context Grounding** e il **Negative Prompting** intervengono come vincoli critici: definiscono esplicitamente i margini di tolleranza dell'interpretazione e vietano tassativamente l'invenzione di entità mediche, ancorando le deduzioni esclusivamente ai segnali acustici forniti. Per affinare ulteriormente la sensibilità del modello verso i rumori tipici dei sistemi ASR, i **Few-Shot Examples** calibrano l'attenzione su pattern anomali specifici (come le frammentazioni o le fusioni articolo-sostantivo), mentre lo **Structured Output** finale in JSON vincola la naturale tendenza discorsiva del modello a uno schema sintattico rigido,

garantendo che il risultato dell'astrazione linguistica si traduca in un formato dati perfettamente robusto e passabile per l'integrazione nella pipeline software.

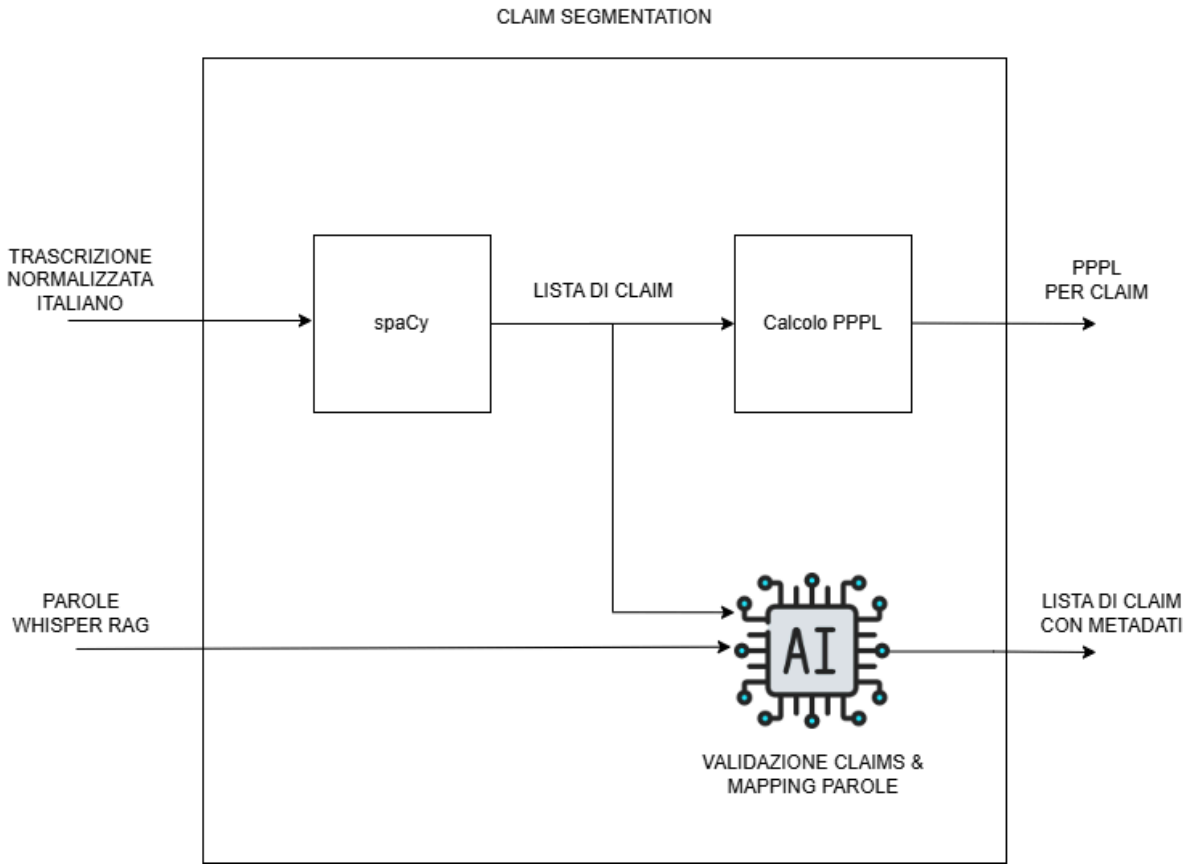
Il prompt definisce una gerarchia esplicita per la risoluzione dei conflitti:
Quando le tre sorgenti convergono, il token viene preservato senza modifiche anche se insolito. Quando Whisper Standard è semanticamente coerente, l'IPA lo supporta e Whisper-RAG propone un termine dialettale improbabile, il modello conclude che il RAG è contaminato dal prior lessicale e Whisper Standard vince. Nella situazione inversa — Whisper Standard produce output insensato, Whisper-RAG ha un termine dialettale coerente e l'IPA supporta la pronuncia dialettale — il modello conclude che Whisper Standard ha fallito l'italianizzazione fonetica e Whisper-RAG vince. C'è poi una regola specifica per l'alta confidenza RAG: nel contesto RAG, un conf_min alto non garantisce correttezza, perché Whisper può copiare vocabolario dal prompt senza reale evidenza acustica. Se l'IPA diverge fortemente, l'IPA ha priorità.

Il prompt contiene tre divieti ripetuti in più punti: non inventare parole, non inventare farmaci o entità cliniche, non forzare correzioni senza evidenza forte. In caso di ambiguità irrisolvibile, il token originale di Whisper-RAG viene preservato. Questo è il comportamento di fallback sicuro: è meglio mantenere una forma dialettale non risolta che produrre una normalizzazione inventata.

L'ultima fase del processo consiste nella determinazione della Pseudo-Perplessità (PPPL) sulla trascrizione normalizzata in lingua italiana. Tale metrica quantifica il grado di anomalia di una frase rispetto a un modello linguistico di riferimento; a tale scopo, è stato impiegato bert-base-italian-cased, un modello specificamente ottimizzato per l'italiano.
Il calcolo del PPPL si basa su una procedura iterativa: per ogni token della frase viene applicata una maschera, richiedendo al modello di stimare la probabilità che la parola originale compaia in quel determinato contesto. I logaritmi di tali probabilità vengono sommati e mediati sul numero totale di token (escludendo i token speciali); il valore risultante viene poi invertito di segno e trasformato mediante una funzione esponenziale per garantire un output esclusivamente positivo.
Per una corretta integrazione nel sistema, il valore ottenuto viene normalizzato in un intervallo compreso tra 0 e 1 utilizzando la tecnica min-max. I parametri di normalizzazione sono stati definiti analizzando la distribuzione dei valori sul campione di 50 audio: il limite inferiore è fissato a 1.66, mentre il limite superiore è stabilito a 30. Quest'ultima soglia è stata calcolata seguendo la formula statistica Q3 + 1.5*IQR (dove l'Intervallo Interquartile rappresenta la differenza tra il 75° e il 25° percentile), garantendo così una robustezza ottimale agli outlier. Qualora il valore calcolato ecceda i limiti definiti, viene applicata un'operazione di CLIP per ricondurlo forzatamente nel range prestabilito.

$$PPPL = exp\left(-\frac{1}{N}\sum_{i=1}^{N} log\left(P(\omega_i|\omega_{\setminus i})\right)\right)$$

5. Claim Segmentation



Il modulo di claim segmentation è necessario per dividere, all'interno di uno stesso periodo, le diverse unità di senso compiuto, chiamate 'claim'. Tale scelta deriva dal fatto che all'interno di una conversazione tra medico e paziente ci possano essere singole frasi contenenti sia domande generiche che descrizioni di sintomi oppure richieste di farmaci. Alla luce di ciò, è stata ritenuta coerente una tale suddivisione, in modo da trattare in maniera differente i vari claim. Infatti non sarebbe corretto dare la stessa importanza ad una semplice domanda o affermazione generale effettuata da un paziente e, ad esempio, ad una descrizione di un sintomo la cui trascrizione sbagliata potrebbe portare conseguenze più serie.
Il modulo di claim segmentation è diviso in tre blocchi funzionali:

1. Modulo spaCy
spaCy è una libreria open-source utilizzata come un analizzatore sintattico. Essa riceve il testo normalizzato in italiano in input ed esegue inizialmente un processo di tokenizzazione, che gli permette di estrarre i singoli token (parole e punteggiatura), ad ognuno dei quali, attraverso un modello statistico basato su reti neurali, assegna una categoria grammaticale. Dopo una operazione di lemmatizzazione che riduce ogni singola parola nella sua forma base (i verbi vengono portati all'infinito, i nomi e gli aggettivi al maschile singolare), viene costruito un albero sintattico stabilendo, quindi, relazioni "padre-figlio" tra le diverse parole. Tale struttura è fondamentale per permettere a spaCy di trovare i nodi principali, identificati come ROOT, attorno ai quali si vanno a costruire i singoli claim, assegnando ad ognuno di

questi nodi le varie parole associate, sfruttando informazioni quali congiunzioni e punteggiatura.
L'output di questo modello è, quindi, una lista di claim, che viene fornita in ingresso al blocco successivo.

2. Allineamento dei claim e propagazione dei metadati
Dopo la segmentazione, il sistema utilizza il modello *Mistral-large* per eseguire simultaneamente due operazioni critiche all'interno di un'unica richiesta. A livello semantico, per ogni claim vengono determinati il *tipo* di appartenenza, la natura interrogativa, l'eventuale presenza di ambiguità e le relative motivazioni. Le categorie previste per la classificazione sono: *"sintomo"*, *"richiesta_farmaco"*, *"condizione_medica"*, *"azione"*, *"domanda_generica"* e *"informazione"*.
Parallelamente, viene effettuato il mapping tra il trascritto generato da Whisper RAG e il claim. Questa operazione produce un array di indici che identifica con precisione quali parole della trascrizione originale compongano effettivamente quel determinato claim. Per garantire l'accuratezza di questo processo, il *system prompt* è stato progettato per imporre regole rigide e un output strutturato in formato JSON.
Infine, per ciascun segmento vengono estratti metadati statistici fondamentali per la successiva analisi del rischio:

- ***conf_min_worst***: individua il token con il livello di confidenza più basso, fungendo da indicatore di criticità;
- ***conf_mean***: rappresenta la confidenza media dei termini che compongono il claim;
- ***conf_geo_period***: calcola la media geometrica della confidenza all'interno del segmento.

3. Calcolo PPPL
Anche a livello di claim viene calcolato il PPPL per il singolo claim individuato, in questo caso i valori di normalizzazione sono specificati tra 1.66 e 160. Questo valore servirà utile come le metriche precedentemente individuate per il calcolo del Risk score

6. Risk Score Calculation

Il modulo legato al calcolo dei rischi per i diversi claim e, di conseguenza, per l'intera trascrizione italianizzata rappresenta il cuore del sistema DAWS. Infatti, esso è il blocco che fornisce le informazioni necessarie per le decisioni future (accettare la trascrizione, chiedere domande delucidative o chiedere di ripetere tutto).
La logica di calcolo avviene a livello di claim e, una volta calcolati i rischi per ognuno di essi, utilizza tali informazioni per effettuare un calcolo sul rischio globale dell'intera trascrizione.
Per calcolare il rischio dei vari claim, il modulo sfrutta le informazioni contenute nella lista prodotta dal blocco di claim segmentation, utilizzandole all'interno di una somma pesata.
É importante notare, però, che i pesi impiegati all'interno del calcolo non sono uguali per ogni claim, ma differiscono in base al loro tipo, in modo da allineare l'analisi alla criticità semantica del contenuto, dato che non tutti i segmenti di una trascrizione hanno lo stesso impatto sulla valutazione finale effettuata dalla pipe. Infatti, in un contesto medico, un'incertezza fonetica su un nome di un farmaco o un sintomo rappresenta un rischio oggettivamente superiore rispetto ad una bassa confidenza su una domanda di carattere generale.
Per questo motivo sono state determinate le seguenti matrici di pesi per i vari tipi di claim:

Tipo di Claim	conf_min_ worst (w1)	conf_mean (w2)	conf_geo (w3)
Richiesta Farmaco	0.30	0.28	0.42
Sintomo	0.25	0.30	0.45
Condizione Medica	0.28	0.30	0.42
Azione	0.22	0.33	0.45
Domanda Generica	0.20	0.35	0.45
Informazione	0.22	0.33	0.45
Default	0.25	0.30	0.45

Il calcolo del risk score per claim avviene in due step: prima si calcola una valore di rischio del trascritto di whisper RAG con tutte le informazioni del claim nel modo seguente:

$$R_{transcript} = (1 - confMinWorst) * w1 + (1 - confMean) * w2 + (1 - conf\,Geo) * w3$$

Successivamente si va ad utilizzare il valore di *pppl* normalizzato del claim come rischio di traduzione, questa ha peso unitario.

$$R_{translate} = PPPL_{claim_norm}$$

Per combinare correttamente queste due aliquote si è definita una matrice dei pesi alpha che in base al tipo di claim, applica una combinazione lineare tra le due componenti facendole pesare in maniera differente.

	richiesta_ farmaco	sintomo	condizione_ medica	azione	domanda _generica	informazione	_default
alpha	0.80	0.70	0.75	0.65	0.65	0.70	0.70

$$R = alpha * R_{transcript} + (1 - alpha) * R_{translate}$$

A questo risultato in realtà si fa un ulteriore "guardia" per dare più importanza alla componente dell'errore di trascrizione che quello di traduzione, ovvero si controlla se il rischio totale è più basso dell'85% del rischio di trascrizione allora bisogna associare come rischio totale quest'ultimo. Ciò serve per prevenire quei casi dove R_translate viene estremamente basso e R viene influenzato troppo da questa componenete.

$$R = max(R;\ 0.85 * R_{transcript})$$

Infine per ogni risk score calcolato per claim viene associato un *"semaforo"* verde, giallo o rosso per indicare generalmente quanto è rischioso quel claim, con esattezza le soglie sono:

- R < 0.35 => Verde
- R < 0.60 => Giallo
- altrimenti Rosso

Ora quello che manca è il calcolo del risk score globale su tutto il periodo, per fare ciò si è deciso di applicare una metodologia basata su regole, controllando la distribuzione dei tipi di rischi tra i vari claim nel periodo.
L'idea di fondo è calcolare in primis, il numero di claim rossi e gialli che ci sono in un periodo e calcolare il peso associato di rossi e gialli. Questo peso si calcola come la somma dei pesi che compaiono tra i vari claim, il peso varia in base al claim come riportato nella tabella seguente. I pesi sono gli stessi se il rischio del claim è giallo o rosso.

	richiesta_ farmaco	sintomo	condizione_ medica	azione	domanda _generica	informazione
w	1.5	1.5	1.5	1	0.8	0.7

I primi tre tipi di claim sono quelli critici, ovvero che sono più importanti da tenere sott'occhio se hanno avuto un rischio associato alto.

Una volta calcolati i pesi per un periodo si calcolano le loro proporzioni:

p_red   = red_weight   / total_weight
p_yellow = yellow_weight / total_weight
p_risky = (red_weight + yellow_weight) / total_weight

Infine si applica una di queste regole in cascata, fino a trovare la prima valida:

1. Ha almeno un claim RED di tipo critico?
   → overall = RED (override immediato, indipendente dalla quantità)

2. Ci sono ≤ 3 claim totali e almeno 1 è RED?
   → overall = RED (pochi claim, non si può "diluire" il rischio)

3. p_red ≥ 20%?
   → overall = RED

4. p_risky ≥ 40% (yellow + red insieme)?
   → overall = YELLOW

5. p_yellow ≥ 25%?
   → overall = YELLOW

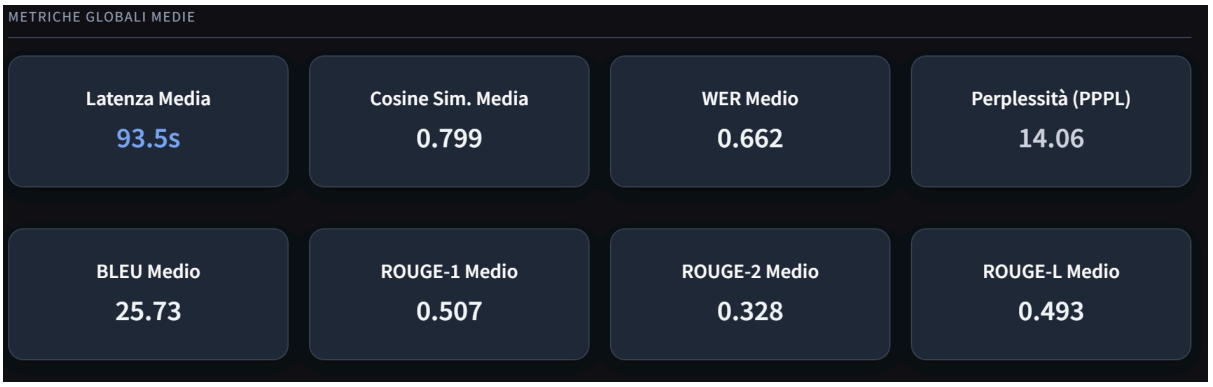
6. Nessuna condizione soddisfatta?
   → overall = GREEN

7. Metriche e risultati

Per valutare la pipe sono state utilizzate le seguenti metriche:

- **Cosine Similarity**: misura il grado di somiglianza semantica tra due testi rappresentati come vettori numerici (embedding). Il valore ottenuto varia generalmente tra -1 e 1, dove valori più elevati indicano una maggiore vicinanza semantica tra i due testi. Questa metrica consente di valutare la similarità di significato anche in presenza di formulazioni lessicali differenti.
- **Word Error Rate (WER)**: è una metrica utilizzata per valutare l'accuratezza di un sistema di riconoscimento vocale automatico. Essa quantifica il numero di sostituzioni, cancellazioni e inserimenti necessari per trasformare la trascrizione generata automaticamente nella trascrizione di riferimento. Valori più bassi di WER indicano una migliore qualità della trascrizione.
- **BLEU-4 (Bilingual Evaluation Understudy)**: è una metrica impiegata per valutare la qualità di un testo generato automaticamente confrontandolo con uno o più testi di riferimento. Il punteggio viene calcolato analizzando la corrispondenza degli *4-grammi* tra i testi e fornisce una misura della precisione lessicale del contenuto generato. Valori più elevati indicano una maggiore somiglianza con il riferimento.
- **ROUGE-1**: misura la sovrapposizione di unigrammi, ossia singole parole, tra il testo generato e il testo di riferimento. La metrica è orientata al richiamo (*recall*) e valuta la capacità del sistema di preservare le informazioni rilevanti presenti nel testo originale.
- **ROUGE-2**: estende l'analisi di ROUGE-1 considerando i bigrammi, ovvero coppie consecutive di parole. Questa metrica permette di valutare non solo la presenza delle parole corrette, ma anche il mantenimento delle relazioni locali e dell'ordine tra le parole nel testo generato.
- **ROUGE-L**: si basa sulla *Longest Common Subsequence* (LCS), cioè la più lunga sequenza di parole condivisa tra il testo generato e quello di riferimento mantenendo lo stesso ordine relativo. Questa metrica è utile per valutare la coerenza strutturale e la conservazione del significato complessivo del testo, anche in presenza di riformulazioni o parafrasi.

I valori medi ottenuti considerando le metriche poc'anzi descritte sono i seguenti:



I risultati medi ottenuti evidenziano prestazioni complessivamente positive della pipeline proposta, soprattutto considerando la complessità del problema affrontato, che combina trascrizione automatica di parlato dialettale, recupero di contesto tramite RAG e traduzione/normalizzazione semantica mediante Large Language Model.

La **latenza media** osservata è pari a **93,5 secondi** per audio. È importante sottolineare che questo valore è fortemente influenzato dall'utilizzo di modelli esterni accessibili tramite API e non riflette esclusivamente il costo computazionale intrinseco della pipeline. I tempi di risposta possono infatti variare sensibilmente in funzione di diversi fattori esterni, quali la congestione dei server che ospitano i modelli, eventuali meccanismi di rate limiting, il carico di richieste concorrenti, la latenza di rete e la lunghezza dell'input inviato. Di conseguenza, la latenza misurata rappresenta una stima realistica del comportamento del sistema in un contesto operativo, ma non costituisce necessariamente un indicatore diretto dell'efficienza algoritmica delle componenti sviluppate.

Dal punto di vista della qualità semantica, la **Cosine Similarity media** pari a **0,799** rappresenta uno dei risultati più significativi. Un valore prossimo a 0,8 indica infatti un'elevata vicinanza semantica tra le traduzioni generate dalla pipeline e le traduzioni di riferimento, suggerendo che il significato delle frasi viene generalmente preservato anche quando vengono utilizzate strutture sintattiche o scelte lessicali differenti.

Il **WER medio** risulta pari a **0,662**. Sebbene il valore possa apparire elevato se confrontato con scenari tradizionali di riconoscimento vocale, la sua interpretazione richiede particolare cautela nel contesto di questo lavoro. La metrica confronta infatti la traduzione generata con una traduzione di riferimento che, in alcuni casi, potrebbe non essere perfettamente allineata al contenuto effettivamente pronunciato nell'audio. Di conseguenza, differenze tra output della pipeline e riferimento non sempre corrispondono a veri errori del sistema, ma possono derivare da discrepanze presenti nei dati annotati. Inoltre, essendo il task orientato alla traduzione e normalizzazione semantica piuttosto che alla semplice trascrizione letterale, il WER tende naturalmente a penalizzare riformulazioni corrette dal punto di vista del significato ma diverse a livello lessicale o sintattico.

La **perplessità media (PPPL)** pari a **14,06** indica che le frasi prodotte risultano generalmente plausibili e linguisticamente coerenti secondo il modello utilizzato per la valutazione. Tale valore suggerisce che la pipeline genera traduzioni caratterizzate da una buona fluidità sintattica e da una struttura linguistica compatibile con l'italiano standard.

Per quanto riguarda le metriche di confronto testuale, il sistema ottiene un **BLEU medio di 25,73**, evidenziando una discreta corrispondenza lessicale con le traduzioni di riferimento. Tuttavia, anche in questo caso il valore deve essere interpretato considerando che il sistema non mira a riprodurre letteralmente il riferimento, bensì a preservarne il significato complessivo. Pertanto, variazioni lessicali o sintattiche semanticamente equivalenti possono determinare una riduzione del punteggio senza rappresentare necessariamente un peggioramento qualitativo della traduzione.

Le metriche **ROUGE** confermano questa interpretazione. In particolare, il valore di **ROUGE-1 pari a 0,507** indica una buona capacità del sistema di recuperare le informazioni principali presenti nel riferimento. Il valore di **ROUGE-2 pari a 0,328** risulta inferiore, evidenziando differenze più marcate nella struttura locale delle frasi e nell'ordine delle

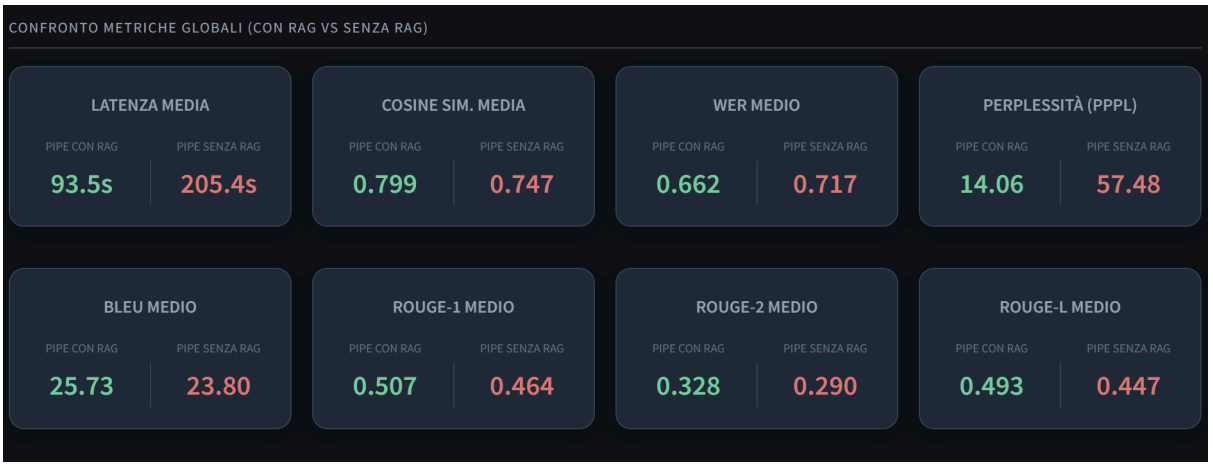
parole. Infine, il valore di **ROUGE-L pari a 0,493** mostra che le principali sequenze informative e la struttura globale del contenuto vengono generalmente preservate.

Nel complesso, l'analisi delle metriche suggerisce che la pipeline riesce a mantenere una buona fedeltà semantica rispetto ai riferimenti, come dimostrato soprattutto dall'elevata Cosine Similarity e dai risultati ottenuti sulle metriche ROUGE. Le metriche maggiormente sensibili alla corrispondenza letterale, come WER e BLEU, risultano invece influenzate sia dalla natura stessa del task di traduzione e normalizzazione, sia dalle possibili imperfezioni presenti nelle traduzioni di riferimento utilizzate per la valutazione. Per questo motivo, nel contesto specifico del presente lavoro, le metriche semantiche assumono un ruolo interpretativo più rilevante rispetto alle metriche basate esclusivamente sul confronto parola per parola.

8. Validazione Whisper RAG e Risk Score

8.1 Validazione Whisper RAG

Al fine di confermare l'efficacia del modulo Whisper RAG, è stata condotta un'analisi comparativa testando una versione della pipeline priva di tale componente. Il confronto si è focalizzato sulle metriche prestazionali individuate in precedenza, portando alla luce i seguenti esiti:



Il confronto tra le due configurazioni evidenzia un miglioramento sistematico delle prestazioni quando viene introdotto il modulo RAG all'interno della pipeline. Tutte le metriche di qualità mostrano infatti valori migliori nella configurazione con RAG, suggerendo che il recupero di contesto semantico fornisca un supporto efficace sia alla fase di trascrizione sia alla successiva traduzione e normalizzazione.

La differenza più evidente riguarda la **perplessità (PPPL)**, che passa da **57,48** nella pipeline senza RAG a **14,06** nella pipeline con RAG. Questa riduzione particolarmente marcata indica che le frasi generate risultano molto più plausibili e coerenti dal punto di vista linguistico. Il recupero di esempi semanticamente simili dal database consente infatti al sistema di disambiguare più correttamente termini dialettali e costruzioni linguistiche complesse, producendo output maggiormente compatibili con l'italiano standard.

Anche la **Cosine Similarity** registra un miglioramento significativo, passando da **0,747** a **0,799**. L'aumento della similarità semantica suggerisce che le traduzioni ottenute mediante la pipeline con RAG preservino in maniera più accurata il significato delle frasi originali rispetto alla configurazione priva di recupero contestuale.

Un comportamento analogo emerge dall'analisi del **WER**, che diminuisce da **0,717** a **0,662**. Sebbene i valori assoluti debbano essere interpretati con cautela per le ragioni già discusse, la riduzione dell'errore evidenzia una maggiore vicinanza tra gli output generati e le

traduzioni di riferimento. Questo risultato suggerisce che il contesto fornito dal RAG contribuisca a ridurre errori di interpretazione e ambiguità lessicali.

Le metriche basate sulla sovrapposizione testuale confermano ulteriormente il beneficio introdotto dal recupero di conoscenza. Il **BLEU** aumenta da **23,80** a **25,73**, mentre tutte le metriche **ROUGE** mostrano incrementi consistenti:

- **ROUGE-1**: da **0,464** a **0,507**;
- **ROUGE-2**: da **0,290** a **0,328**;
- **ROUGE-L**: da **0,447** a **0,493**.

Questi miglioramenti indicano che la pipeline con RAG riesce a recuperare una quota maggiore delle informazioni presenti nei riferimenti e a preservare meglio sia le relazioni locali tra le parole sia la struttura complessiva delle frasi.

In merito all'analisi delle **latenze medie**, il confronto non produce risultati statisticamente rilevanti. Si osserva, infatti, un paradosso per cui la pipeline priva del modulo RAG registra tempistiche sensibilmente superiori, un fenomeno attribuibile a variabili esogene al sistema che ne compromettono la validità come metrica di valutazione comparativa. Indicazioni più attendibili emergono dai timestamp relativi alla fase di trascrizione: questi evidenziano un marginale vantaggio in termini di velocità per la configurazione senza RAG, sebbene tale differenza sia considerata trascurabile ai fini prestazionali complessivi.

Nel complesso, il confronto evidenzia come l'integrazione del modulo RAG produca benefici su tutte le metriche considerate. I miglioramenti più rilevanti si osservano nelle metriche semantiche e di plausibilità linguistica, confermando che l'accesso a conoscenza contestuale rappresenta un elemento determinante per affrontare efficacemente il problema della trascrizione e traduzione di parlato dialettale.

8.2 Validazione modulo risk score

Per la validazione del risk score, o meglio dei pesi utilizzati e le soglie scelte per i vari livelli, si è scelto di estrarre un campione casuale di 20 dalla popolazione degli audio ed etichettarli manualmente. L'operazione è stata divisa in due fasi: valutare sia l'overall_risk ovvero il livello assegnato a tutta la trascrizione (verde giallo o rosso), sia il livello di score di ogni claim. Si è assegnato un voto da 0 a 5 e poi si è fatta la media sui 3 annotatori. Questi sono stati i risultati ottenuti.

	L	A	M
medie globali	3.45	3.35	3.50
medie locali	3.52	3.53	3.65

Si è valutato anche per i singoli annotatori quante sono state le occorrenze dei valori da 0 a 5 per le votazioni sia dei singoli claim che quelle globali:

	L	A	M
0_G	1	2	2
1_G	3	1	1
2_G	1	3	2
3_G	3	1	3
4_G	5	8	4
5_G	7	5	8
0_L	2	2	2
1_L	5	3	3
2_L	8	9	8
3_L	6	4	7
4_L	6	11	4
5_L	17	15	20

Il seguente risultato porta alle seguenti considerazioni: che il sistema di calcolo di risk score globalmente, sui termini medi, produce delle valutazioni coerenti anche con delle istanze sbagliate o poco attendibili.
A livello delle valutazioni sulle singole occorrenze sia per il risk score globale ma anche quello del risk score sui singoli claim, tutti e tre gli annotatori hanno individuato un numero di occorrenze dei valori 4 e 5 insieme che superano la metà delle occorrenze previste, (44 per i singoli claim e 20 a livello globale). In sostanza hanno avuto la percezione di una buona affidabilità dei risultati proposti seppur non sempre perfetta.

9. Dashboard

Per la visualizzazione e l'analisi dei risultati prodotti, è stata creata un'applicazione web sviluppata con il framework Streamlit in python. L'interfaccia è strutturata come applicazione multipagina: una pagina di home(app.py) e quattro sezioni accessibili da una sidebar ognuna implementata come file autonomo.

La struttura è la seguente :

```
dashboard/
├── app.py
├── db.py
├── requirements.txt
├── .env
├── .streamlit/
│   └── config.toml
├── pages/
│   ├── 1_overview.py
│   ├── 2_dettaglio.py
│   ├── 3_aggregata.py
│   ├── 4_upload.py
│   └── 5_confronto_metriche.py
└── utils/
    ├── style.py
    └── audio.py
```

- **./app.py** è la pagina di home in cui si imposta la configurazione globale e viene mostrato il nome della dashboard.
- **./db.py** centralizza tutte le query a Mongodb, infatti le pagine non interrogano direttamente il database ma importano solo le funzioni esposte da questo modulo. Vengono coinvolte le principali collection del db: Sessions, partecipants, Transcripts, Risk_Logs, Pipeline_Stages, Transaltion_Metrics, recordings, da cui si estraggono tutti i dati da mostrare. Il layer db.py supporta l'accesso a database distinti sullo stesso cluster Atlas, mantenendo un'unica connessione condivisa.
- **page1 Overview**: si mostra una panoramica generale su tutte le sessioni completate. i KPI mostrati sono: numero totale di sessioni processate; distribuzione dei livelli di rischio; confidence media. Vengono poi mostrati dei grafici fatti tramite la liberia Plotly: Scatter plot temporale dei tempi di processing per sessione, Istogramma raggruppato della distribuzione dei livelli di rischio per categoria di prompt, bar chart del risk score per singolo file audio. In fondo alla pagina infine è presente una tabella delle sessioni recenti.
- **page2 Dettaglio**: qui è possibile ispezionare approfonditamente una singola sessione, selezionata tramite un dropdown. La pagina è strutturata in 5 sezioni. Nella prima sezione vengono mostrati i metadati demografici della persona, e viene data la possibilità di ascoltare l'audio scaricato da GoogleDrive usando il drive_file_id memorizzato in recordings, e di leggere la trascrizione originale associata

all’audio.Nella sezione 2 si espongono 3 livelli di elaborazione del testo: prima la trascrizione grezza di whisper large, poi la trascrizione di whisper large abbinato alla tecnica RAG e poi la versione tradotta del testo dall’llm. Per il testo del livello 2 vengono evidenziate le parole semanticamente problematiche segnalate dall’LLm, con la possibilità di mostrare la motivazione di esso al passaggio del cursore sulla parola. La terza sezione della pagina mostra il rischio complessivo e il dominio semantico. La quarta sezione è dedicata all’analisi dei claim in cui ogni claim viene mostrato in un expander. Il titolo dell'expander riporta risk level, tipo, score e anteprima del testo; i claim RED sono espansi di default. All'interno: il testo del claim, il verbatim audio originale, e due colonne con barre progress per i segnali ASR che rappresentano le metriche di valutazione della trascrizione, e le metriche riferenti alla traduzione. Vengono poi mostrati i tempi di esecuzione di ogni stage della pipe, mettendo in risalto i tempi piu lunghi tramite delle barre.La pagina finisce con le metriche di valutazione associate, ovvero tempo di esecuzione ,cosine similarity, WER, BLEU score, ROUGE-1, ROUGE-2, ROUGE-L, PPPL.

- **page3 Aggregata**: ha come obiettivo mostrare i risultati cross-session.Troviamo 8 KPI calcolati sulle sessioni: latenza media, cosine-similarity, WER medio, BLEU medio e ROUGE1 medio, ROUGE-2 medio, ROUGE-L medio, PPPL medio. Ci sono poi Donut chart Plotly della distribuzione dei livelli di rischio; bar chart orizzontale del tempo di calcolo medio per categoria semantica di prompt. Per quanto riguarda l’analisi dialettale invece si mostrano in un bar chart le top 15 parole problematiche per conteggio, e un donut chart della distribuzione dei tipi dialettali. Infine un Heatmap che mostra rischio per tipo di claim e una tabella riassuntiva avanzata che per ogni sessione filtrata espone: filename, dialetto, risk level (colorato), tutte le metriche, numero totale di claim e numero di claim rossi. Inoltre è possibile tramite una barra laterale filtrare i risultati tramite dei filtri demografici.
- **page4 Upload**: l’ultima pagina offre un interfaccia per inviare un nuovo file audio alla pipline di inferenza remota su kaggle e salvare i risultati sul database. L'utente inserisce l'URL Ngrok del notebook Kaggle attivo,compila i metadati demografici del paziente (fascia d'età, genere, dialetto, contesto abitativo) tramite selectbox e carica il file audio (.wav o .mp3) con anteprima player oppure registra il file audio tramite browser. Al click su "Avvia Inferenza su Kaggle GPU", la pagina invia una richiesta HTTP POST all'endpoint Flask/FastAPI esposto da Kaggle, passando il file audio e i metadati. In caso di risposta 200, il session_id ricevuto viene salvato e l'utente viene reindirizzato automaticamente alla pagina di dettaglio. Non avendo a disposizione il trascritto originale dell’audio non è possibile ovviamente calcolare le varie metriche di valutazione.
- **page5 confronto risultati:** ha l’obiettivo di comparare i risultati con quelli ottenuti dall’esecuzione della pipeline senza l’utilizzo della tecnica RAG per validare tale scelta progettuale. Si mostrano tutte le metriche di valutazione precedentemente esposte e i risultati ottenuti.
- **style.py** : definisce il css.
- **audio.py**: espone un’unica funzione ceh scarica un file audio pubblico da Google Drive tramite endpoint. Gestisce il token di conferma antivirus che goolge impone per file di grandi dimensioni e restituisce i bytes del file.

10. Trascrizione in tempo reale

Come parte finale del lavoro si è scelto di implementare una piccola demo per effettuare in tempo reale l’elaborazione di un audio tramite tutta la pipe per mostrarne il funzionamento e valutarne i risultati. L'architettura della trascrizione in tempo reale si basa su tre componenti distinti che lavorano in sequenza: un server HTTP esposto da Kaggle, un tunnel pubblico per renderlo raggiungibile dall'esterno, e la dashboard Streamlit che funge da client. FastAPI è il framework scelto per esporre la pipeline ASR come endpoint HTTP all'interno del notebook Kaggle. L’applicazione FastAPI viene eseguira tramite Uvicorn che è il server ASGI. Dato che kaggle esegue i notebook in ambienti isolati senza un indirizzo IP pubblcio accessibile dall’esterno, si risolve questo problema con Ngrok aprendo un tunnel HTTP dalla porta locale 8000 del container verso sun URL pubblico temporaneo.La libreria pyngrok è il wrapper Python che permette di fare tutto questo dall'interno del notebook senza installazioni manuali: ngrok.connect(8000) apre il tunnel e restituisce l'URL pubblico che viene poi incollato nella pagina 4 della dashboard. La liberira librosa invece serve per caricare il file audio in memoria come array NumPy, ricampionandolo automaticamente a 16.000 Hz. Questa frequenza di campionamento è il sample rate nativo su cui Whisper è stato addestrato, e passare un audio a frequenza diversa degraderebbe le prestazioni della trascrizione. Librosa gestisce in modo trasparente tutti i formati audio supportati (WAV, MP3, ecc.) e il resampling, sollevando la pipeline dalla necessità di gestire questa normalizzazione manualmente.

11. Migliorie per il futuro

- Avendo dimostrato l'utilità e la validità del sistema RAG, potrebbe essere interessante implementare una logica di alimentazione, dove con le nuove trascrizione e traduzioni prodotte dalla pipeline, questo vengono aggiunte alla collezione vettoriale del sistema RAG, ovviamente qui potrebbe causare una perdita di qualità… è una miglioria che potrebbe essere valida però introducendo anche qualche logica di penalizzazione per questi nuovi testi aggiunti.
- La vera parte delicata del sistema è il calcolo del risk score per claim, che è estremamente sensibile alle matrici dei pesi e le soglie determinate. Un'idea per il futuro sarebbe quello di realizzare un modello di ML per trovare la configurazione ottima di pesi, come un ottimizzatore Bayesiano. Ciò richiede nell'usare una ground truth realizzata ad hoc e probabilmente un campione di dati più ampio.